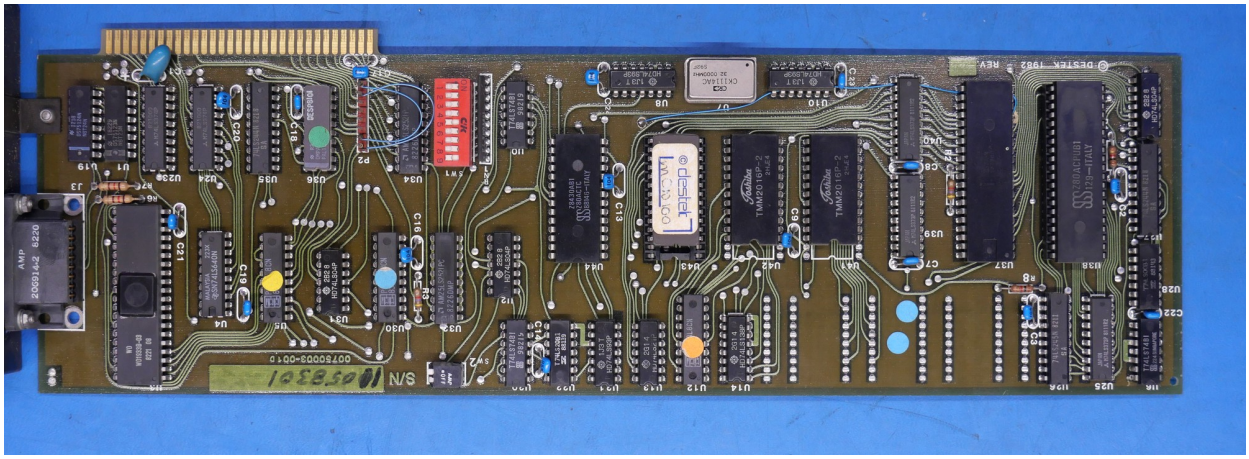
The Destek 1982 ISA Network Card

A ROM-level analysis of a pre-Ethernet IBM PC local area network adapter

and of the MAGIX network-boot firmware found with it



Board 00750003-001D · S/N 10058301 · silkscreen © DESTEK 1982
Firmware `mdn100`, © 1984 The Destek Group, Inc.

Source material: Internet Archive item `destek-isa-network-card`
`https://archive.org/details/destek-isa-network-card`

Contents

1 Summary **2**

2 The material **3**
2.1 A caveat about the supplied disassembly . . . 3

3 The hardware **3**
3.1 Component inventory . . . 4
3.2 The Z80's I/O map . . . 4

4 The card firmware **6**
4.1 Interrupts: a CTC used as a vector generator . . . 6
4.2 Link layer . . . 6
4.3 Media access: carrier sense, collision, backoff . . . 6
4.4 Frame and buffer layout . . . 7
4.5 Buffers, and the main loop . . . 7
4.6 Statistics, and a management protocol the host never sees . . . 8
4.7 Two defects . . . 8

5 The MAGIX boot ROM **9**
5.1 One image, two jobs . . . 9
5.2 Self-relocation . . . 10
5.3 Screen . . . 10
5.4 Finding the card . . . 10
5.5 The other four adapters, and what "9600" means . . . 11
5.6 Booting . . . 12
5.7 A forty-year-old bug, preserved . . . 12

6 What this was for **13**

A Protocol specification **14**
A.1 Physical and framing layer . . . 14
A.2 Addressing . . . 14
A.3 Packet format . . . 14
A.4 Station-status protocol . . . 15
A.5 Host interface . . . 16
A.6 The configuration block . . . 17

B Building the server side today **17**
B.1 The line is an IBM System/360 party line, not a serial link . . . 18
B.2 The line rate is far lower than it looks . . . 18
B.3 What to measure first . . . 19
B.4 Choosing a transceiver . . . 19
B.5 The server software . . . 20

C Things worth demonstrating **21**

D Method, and how to check this **22**
D.1 Provenance . . . 22
D.2 Tools . . . 23
D.3 Verification . . . 23
D.4 Repository layout . . . 23
D.5 What was *not* done . . . 23

E Open questions, and what is known about the company **23**
E.1 Questions the ROMs cannot answer . . . 23
E.2 Parts of the ROMs still undecoded . . . 24
E.3 The Destek Group . . . 24

1 Summary

The Destek card is not a network interface in the modern sense. It is a complete **Z80A single-board computer that happens to live on the ISA bus**: its own CPU, its own 4 KiB of ROM and 4 KiB of RAM, its own DMA controller, its own interrupt controller, and a Western Digital WD1933B-03 Synchronous Data Link Controller driving a DB-15 onto a shared twisted-pair line. The host PC never touches the wire. It hands whole packets to the card through a two-byte-wide mailbox and gets an interrupt back when one arrives.

The firmware in that card — `mdn100`, © 1984 The Destek Group — implements a **CSMA/CD packet network over HDLC/SDLC framing**: carrier sense before every transmission, collision detection, randomised backoff, a five-attempt retry limit, hardware CRC-16, hardware destination filtering, broadcast, nine saturating traffic and error counters, and a station-status protocol that the card answers *by itself*, without the host PC being involved at all. In 1982 that was a serious piece of engineering; 3Com's first EtherLink shipped that same year.

The second ROM in the archive is a different product entirely, and it explains what the card was *for*. **MAGIX v2.20**, © 1985 Sabaki Enterprises, is a diskless-workstation boot ROM. It probes five different LAN adapters, tells whichever it finds what station number to be, downloads a boot image from a fixed server in 128-byte blocks, and jumps into it. The EPROM is hand-labelled `STA-36` — and `36h` is literally the byte at ROM offset 7, the station number this particular workstation announced. Every machine on that network had its own individually burned ROM.

So: this is the hardware half of an early-1980s diskless-PC network, of the kind that let an office run a dozen IBM PCs off one file server before hard disks were affordable in every desk.

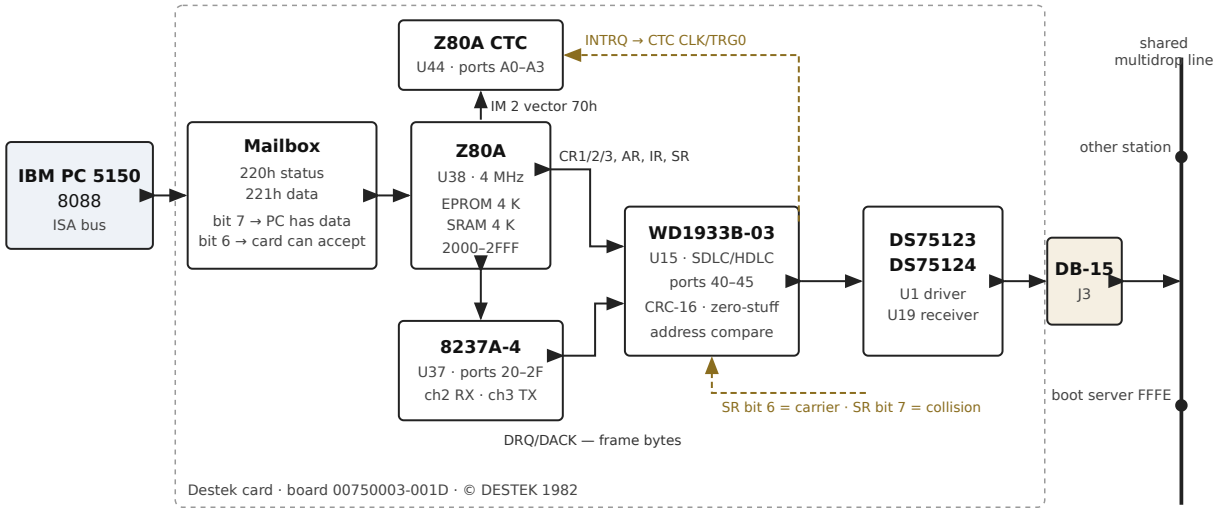


Figure 1: Functional block diagram. The host's only contact with the network is the mailbox on the left; everything to the right of it is the card's own computer. The dashed amber paths are the two places where the design is unusual: the WD1933's interrupt request is routed through a Z80 CTC channel to synthesise a mode-2 vector, and two modem-control inputs on the WD1933 are repurposed as carrier sense and collision detect.

At a glance

ISA I/O ports	`36E/36F` (IRQ 7) or `220/221` (IRQ 3). Even port = status register and card-reset strobe; odd port = data.
This card's straps	decode to base `220h`, IRQ 3.
Z80 memory map	ROM `0000`–`0FFF`, SRAM `2000`–`2FFF`, sized automatically at boot.
Wire format	HDLC: FLAG · A · C · I-field · FCS · FLAG, CRC-16 in hardware.
Packet header	destination (3) + source (3) + type (1); `FFFF` = broadcast.
Maximum payload	four times a host-supplied parameter P; the packet limit is $4P + 7$. MAGIX configures $P = 34$, so 136 bytes of payload.
Media access	carrier sense, collision detect, randomised backoff, 5 retries.
Boot server	station `FFFE` (SDLC A-field `FEh`).
Boot image	128-byte blocks, 8-deep window, loaded at `0040:0000`.
Station number	burned into each workstation's own boot EPROM.

2 The material

The Internet Archive item contains nine original files. Three matter for this analysis, and one of them is mislabelled in a way worth stating plainly up front.

File	Size	What it actually is
`Destek MON100.BIN`	4096 B	The card's own Z80 firmware, from the 2732 EPROM at U43. Code occupies `0000`–`0701`; the rest is erased.
`Card ROM.asm`	144 kB	A disassembly of the above. See the caveat below.
`5150 Option ROM.bin`	8192 B	*Not* a Destek ROM. MAGIX v2.20, Sabaki Enterprises 1985 — the network boot ROM from the host PC's motherboard.
`Card Front.JPG`, `Card Back.JPG`	7 MB each	Board photographs, 5184 × 2920.
`5150 Option ROM.png`	0.9 MB	The boot EPROM *in situ*, with its hand-written label.
`Notes.txt`	734 B	The uploader's parts list and switch settings.

2.1 A caveat about the supplied disassembly

`Card ROM.asm` renders the range `03AE`–`0450` as raw `defb` bytes. That range is the card's *entire interrupt service routine* — the code that actually moves packets. It was missed because it is unreachable by static control flow: it is entered only through the Z80 mode-2 vector table at `0070`, which the uploader's tool did not follow. Every statement in this report about receive handling, collision detection and DMA re-arming comes from re-disassembling that block. The script `tools/disassemble.sh` in the accompanying repository regenerates it.

3 The hardware

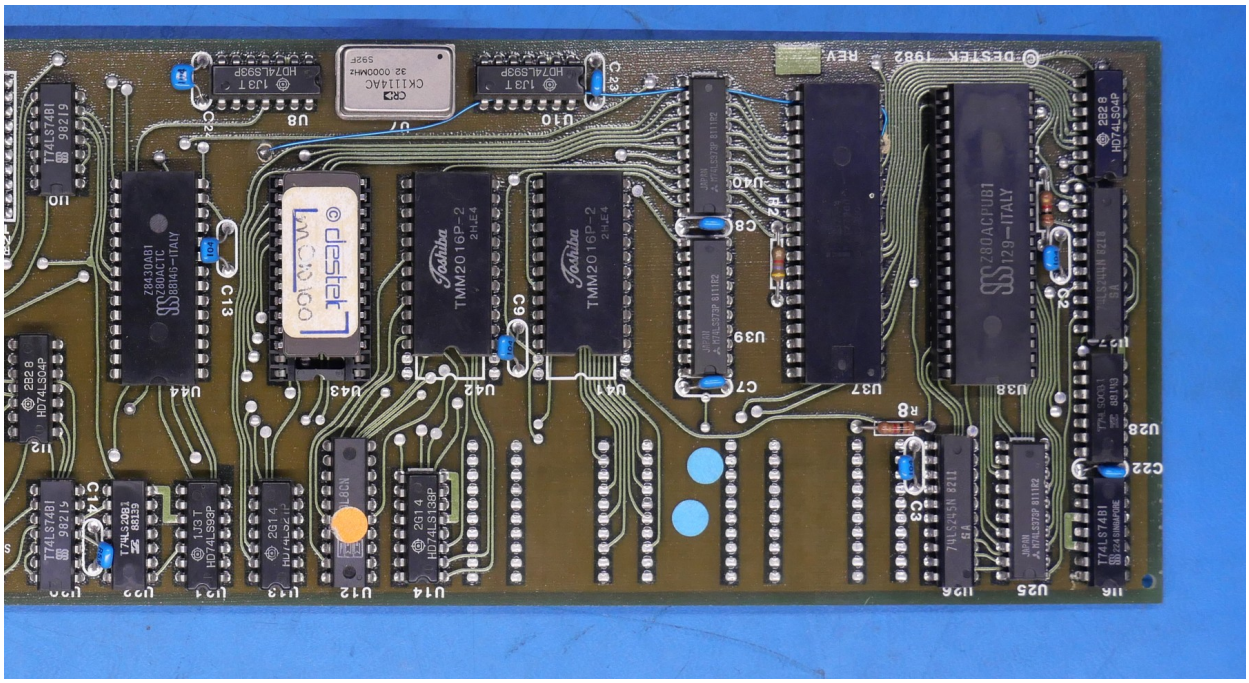


Figure 2: The computer half of the card. Left to right: U43, the 2732 EPROM under a hand-written *destek mon100* label; U44, an SGS Z8430 Z80A CTC; the two Toshiba TMM2016P-2 static RAMs; the pair of M74LS373 address latches for the DMA controller; U37, the 8237A-4; and U38, an SGS Z80ACPU. Above them, the 32.000 MHz oscillator (U7) and two 74LS93 dividers.

3.1 Component inventory

Ref	Part	Role
U38	SGS Z80ACPUB1	Z80A CPU
U43	2732 EPROM	the `mdn100` firmware
U41, U42	Toshiba TMM2016P-2	2 KiB SRAM each; 4 KiB total at `2000`
U44	SGS Z8430AB1	Z80A CTC — used purely as an interrupt vector generator
U37	Intel 8237A-4	DMA; channel 2 receives, channel 3 transmits
U39, U40	M74LS373P	DMA address latches
U15	**WD1933B-03**	Western Digital SDLC/HDLC controller, date code 8221
U1	DS75123N	dual TTL-to-MOS line driver
U19	DS75124N	triple line receiver
U4	SN74LS640N	inverting octal transceiver
U34	Am25LS2521PC	8-bit equality comparator — the ISA I/O address decoder
U36	PAL, marked `DESP8101`	Destek's custom glue logic
U7	CRE CKI114AC	32.0000 MHz oscillator
U8, U10	HD74LS93	clock dividers
J3	AMP 209914-2	DB-15 network connector
SW1	9-position DIP	I/O address select
SW2	2-position DIP	interrupt select
P2	8-pin header	option straps (1–8 and 2–7 wired on this card)

3.2 The Z80's I/O map

Derived from every `IN` and `OUT` instruction in the firmware, and cross-checked against the WD1933 datasheet.

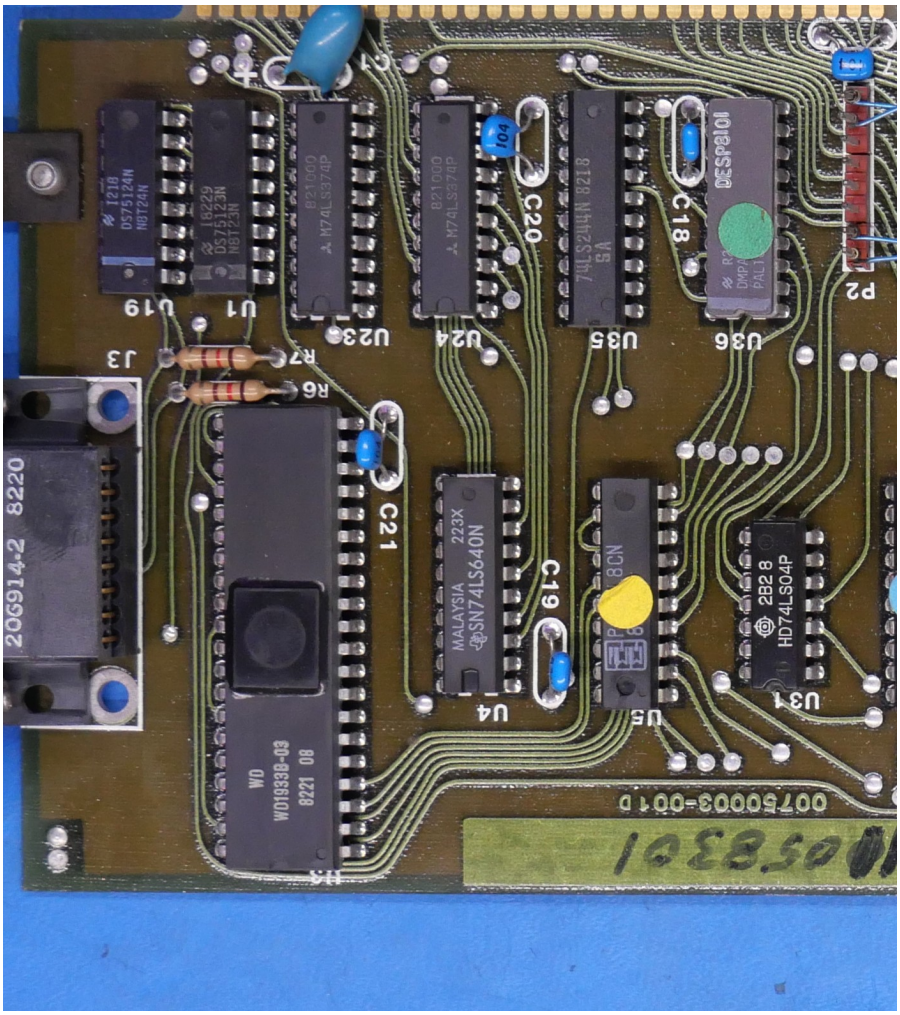


Figure 3: The network half. U15 is the WD1933B-03 with a 1982 date code (8221), sitting directly behind the DB-15 at J3. U4 is an SN74LS640 transceiver. Off the top of the frame are U1, a DS75123N dual line driver, and U19, a DS75124N triple line receiver — the parts that put the signal on, and take it off, the shared line.

Port	Device	Use
20–2F	8237A-4	ch. 2 = receive (address 24, count 25); ch. 3 = transmit (26, 27); 28 command, 2B mode, 2C clear byte-pointer, 2D master clear, 2F mask
40	WD1933 CR1	bit 7 ACT REC, bit 6 ACT TRAN (raises RTS), bit 0 MISC OUT (pulsed as a board strobe)
41	WD1933 CR2	written 41h
42	WD1933 CR3	written 00h
43	WD1933 AR (write)	this station's address, for the hardware comparator
44	WD1933 IR (read)	interrupt cause
45	WD1933 SR (read)	status — including carrier and collision
60	write-only strobe	fires when the card becomes ready to accept a packet from the PC
80	read-only	DIP switches; fallback station address
A0–A3	Z80 CTC	four channels
C0	mailbox status	bit 7 = host wrote a byte; bit 6 = host read the last byte
C1	mailbox data	the bidirectional 8-bit path to the ISA side

There is deliberately *no* CPU-visible data port on the WD1933: frame bytes never pass through the Z80. The receiver and transmitter holding registers are driven entirely by the 8237 through the chip's DRQI/DRQO and DACK lines, which is why only control and status registers appear in the map.

4 The card firmware

4.1 Interrupts: a CTC used as a vector generator

The WD1933 is not a Zilog peripheral. It cannot place an interrupt vector on the bus, so it cannot take part in a Z80 mode-2 daisy chain. Destek's solution is neat: the WD1933's `INTRQ` pin is wired to **CTC channel 0's CLK/TRG input**, and that channel is run as a counter with a time constant of 1 — so every interrupt edge from the SDLC controller produces exactly one Z80 mode-2 interrupt, with the CTC supplying the vector.

```
0620 SUB14:
     ld   a,03h ; out (A0h),a ... out (A3h),a   ; reset all four CTC channels
     ld   hl,0070h
     ld   a,l   ; out (A0h),a                   ; ch0 interrupt vector = 70h
     ld   a,h   ; ld i,a ; im 2                 ; I = 00h -> vector table at 0070h
     ld   a,D7h ; out (A0h),a                   ; IE, counter mode, rising edge, TC follows
     ld   a,01h ; out (A0h),a                   ; time constant 1 -> interrupt every edge
     reti                                       ; RETI, not RET: clears the daisy chain
```

The vector table at `0070` is `AE 03 | 78 00 | 78 00 | 78 00`: channel 0 goes to the real handler at `03AE`; channels 1–3 go to a stub at `0078` that is just `ei / reti`. `0000`, the `RST 38` entry at `0038` and the NMI entry at `0066` all read `di / jp 0514` — any stray interrupt restarts the card.

The handler's first instruction is worth singling out:

```
03AE:  call 0079h        ; 0079h contains ED 4D, i.e. RETI
```

It *calls* a `reti`. Fetching the `ED 4D` opcode pair is what clears the interrupt-under-service latch inside a Z80 peripheral; the `reti` then immediately returns to the caller. The daisy chain is released while the handler carries on running. This is a well-known Z80 idiom, and seeing it here is a good sign that whoever wrote this firmware knew the part properly.

4.2 Link layer

`SUB12` at `05F2` configures the SDLC controller:

```
05F2:  out (40h),01h ; out (40h),00h   ; pulse CR1 bit 0 (MISC OUT) - board strobe
       in  a,(44h)                     ; read IR: clears pending interrupt bits
       ld  a,41h ; out (41h),a         ; CR2
       ld  a,00h ; out (42h),a         ; CR3 - no residual transmit character
       ld  a,(2040h) ; out (43h),a     ; AR = this station's address
```

Decoding `CR2 = 41h` against Figure 7 of the datasheet:

Bit	Name	Value	Meaning
0	AUTO FLAG	1	send continuous flags between frames rather than idling
1	SELF TEST	0	off
2	SDLC LOOP MODE	0	**not** a ring — this is a bus
3–4	RCL0/1	00	eight bits per received character
5	EXT ADDR	0	a single 8-bit address character
6	ADDR COMP	1	**hardware address recognition on**
7	EXT CONTROL	0	one control character per frame

So frames are ordinary HDLC — FLAG, address field, control field, information field, frame check sequence, FLAG — with CRC-16 generated and verified in hardware, zero-insertion and deletion in hardware, and the receiver raising a data request only for frames whose address field matches the address register *or* equals the global address `FFh`.

The DMA controller is set up once, in `SUB13` at `060A`: master clear, command `20h` (extended write), channel 2 mode `06h` = write transfer (peripheral to memory, i.e. receive), channel 3 mode `0Bh` = read transfer (memory to peripheral, i.e. transmit), both in demand mode with address increment.

4.3 Media access: carrier sense, collision, backoff

This is the part that makes the card interesting. `SUB04` at `02E0` is the attempt to seize the line:

```
02E0 SUB04:
     ld  b,40h ; ld c,40h
     in  a,(45h)          ; SR - the WD1933 status register
     and b                ; SR bit 6 = CARRIER DETECT
     ret nz               ; carrier present: someone else is talking, give up
     out (c),b            ; out (40h),40h -> CR1 ACT TRAN: assert RTS, start the frame
     ld  a,07h ; out (2Fh),a   ; unmask DMA channel 3
     xor a ; ret
```

That is carrier sense, plainly. The collision indication is SR bit 7 — nominally the WD1933's Ring Indicator input, repurposed by Destek as a "transmit failed" line from the analogue front end. The interrupt handler reads it:

```
041A:  ld a,b ; and 80h ; jp nz,0423h   ; SR bit 7 -> failure flag = 80h
0420:  ld a,c ; and 10h                 ; else IR bit 4 = XMIT complete WITH UNDERRUN
0423:  ld (2070h),a                     ; transmit-failure flag
       ld a,03h ; ld (206Fh),a          ; TX state 3 = "finished, inspect the result"
```

and SUB05, the transmit state machine, retries — up to five times — after a pseudo-random delay taken from the Z80's DRAM refresh register:

```
0352:  ei
       ld a,r          ; the refresh counter, used as a cheap random number
       add a,a
       add a,7Dh
       ld b,a
       ld c,01h
       jr nc,035Fh
       inc c           ; carry -> roughly double the delay
035F:  djnz 035Fh
       dec c
       jp nz,035Fh
```

Carrier sense, collision detect, randomised backoff, a bounded retry count: this is CSMA/CD, on a card whose silkscreen says 1982.

4.4 Frame and buffer layout

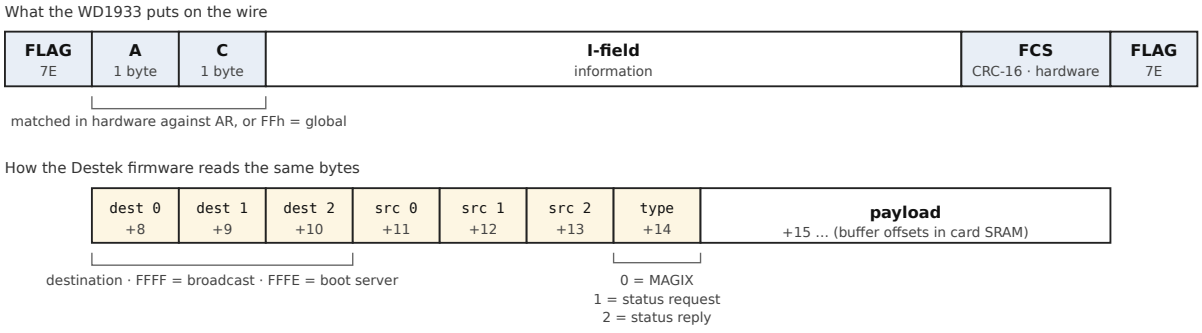


Figure 4: The HDLC frame and the Destek header mapped onto it. The controller DMAs the address, control and information fields into card RAM, starting at offset +8 of a buffer; the frame check sequence is verified in hardware and never reaches memory.

The trick worth admiring is that the SDLC address and control fields *are* the low two bytes of the packet destination. A broadcast is destination FFFF, whose address field is therefore FFh — the WD1933's hardware global address, which the chip accepts without any extra configuration. One design decision buys both unicast filtering and broadcast, for free.

4.5 Buffers, and the main loop

LBL2 at 0514 is the reset path: clear 2000–20FF, set the stack pointer, run the host handshake, then **size the SRAM** by writing FF and 00 upward from 2097 until the readback fails. Whatever is found is carved into

fixed-stride buffers by SUB10, using a 16-bit divide routine at 063F to work out how many fit. With MAGIX's 153-byte stride that is 25 buffers, occupying 2097–2FF7 and leaving 120 bytes spare.

Those buffers move between **three singly-linked lists and one ring cursor**. This is the data structure the whole firmware is built on, and it is what anyone writing an emulator will need:

Head	List	Discipline
2053	free pool	5 buffers. Push SUB08 (0491), pop SUB09 (04A1); the "is it empty" test reads only the high byte, at 2054.
205E	card → host	a queue header {next, tail}. Append SUB06 (0451), pop SUB07 (0467), initialised by SUB22 (06EE).
2071	card → network	the same discipline and the same routines.
206B	receive ring cursor	20 buffers chained head to tail, advanced by the ISR.

SUB07 holds the one subtlety: when it empties a queue it repoints the tail back at the header itself (0480–0489), which is what makes the next append work.

The main loop is cooperative and has no preemption beyond the interrupt handler:

```
00B5 LBL1:
     call SUB03      ; arm receive DMA on channel 2, activate the receiver
00B8:
     call SUB01      ; PC -> card       (host is writing a packet to send)
     call SUB02      ; card -> PC       (deliver a received packet)
     call SUB05      ; card -> network  (the transmit state machine)
     jp   00B8h
```

Each of the three is a non-blocking state machine with its own state byte, so a slow host or a busy line never stalls the other two. The real work happens in the interrupt handler, which acknowledges the controller, latches the interrupt and status registers into the current buffer, reads the residual DMA count to compute the received length, marks the buffer full, advances the ring and re-arms channel 2 — all in the alternate register set, entered with `exx / ex af,af'`.

4.6 Statistics, and a management protocol the host never sees

Bytes 2079–2096 hold a 30-byte station-status block: the six-character ROM string mdn100 followed by nine counters. They saturate rather than wrap — the increment routines back the value out again on overflow.

Address	Width	Counts
207F	32	frames transmitted successfully
2083	32	frames received and processed
2087	32	frames received but addressed to another station
208B	16	CRC errors
208D	16	overruns and bad residual lengths
208F	16	aborted or invalid frames
2091	16	transmit retries after collision or underrun
2093	16	receive overruns — no free buffer in the ring
2095	16	packets from the host rejected for a bad length

A frame of **type 1** is a status request. The card answers it entirely on its own, building a 37-byte **type 2** reply containing that block and addressing it back to the requester. The host PC is never told the exchange happened.

That means any station could poll any other for its identity and error counters, and get an answer even if the target machine's software had crashed, or had never been loaded at all. It is a deliberate network-management feature, and — as Appendix C argues — it is the single most useful thing about this card for anyone bringing it back to life today.

4.7 Two defects

A dead store in the error path. Two different startup failures were meant to report different codes to the host. One of them cannot:

```
05DA:  ld a,01h        ; bad station address
       jp 05E1h
05DF:  ld a,07h        ; buffer parameter < 8  --  immediately clobbered
05E1:  ld a,01h        ; <- both paths arrive here
       out (C1h),a
       halt
```

Both failures therefore report `01`. Since the PC-side ROM only tests for non-zero before printing `!!! LAN BOARD FAILED !!!`, nothing observable changes — but code `07` was clearly intended. Verified against the raw bytes `3e 01 c3 e1 05 3e 07 3e 01 d3 c1 76`.

A near miss that turns out to be fine. `SUB09` reads the single byte at `2054` and treats zero as "no free buffer". Nothing ever writes `2054` — which looks fatal, until you notice that `2053` holds a 16-bit free-list head pointer and `2054` is its *high byte*. Since every valid buffer address is at least `2097`, testing the high byte for zero is a correct and very cheap empty-list check. It is recorded here because the supplied disassembly labels the two bytes separately, which invites exactly this misreading.

5 The MAGIX boot ROM

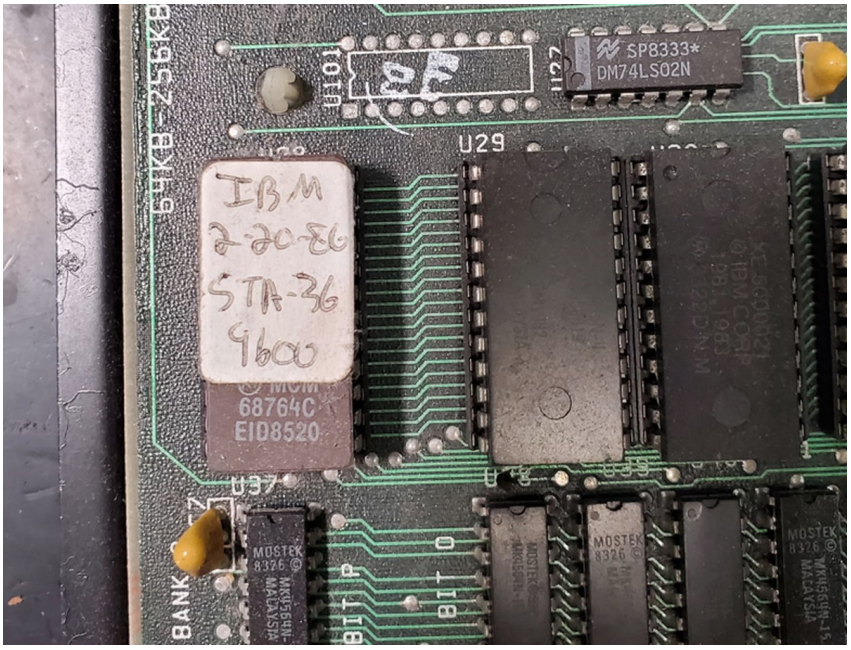


Figure 5: The MAGIX EPROM in the host PC's spare motherboard ROM socket, next to the IBM BIOS ROMs. The hand-written label reads `IBM / 2-20-86 / STA-36 / 9600`. `STA-36` is the station number, and `36h` is exactly the byte at offset 7 of the image — the value the ROM hands to the network card. The machine displayed it in decimal, as `ADDRESS: 054`.

5.1 One image, two jobs

The file is a valid PC option ROM: `55 AA`, a length byte of `10h` (sixteen 512-byte blocks), an 8-bit checksum of zero, and an entry point at offset 3 that jumps to offset `49h`. The 5150's power-on self test finds it during its option-ROM scan.

But the last five meaningful bytes of the image sit at offset `1FF0`:

```
1FF0:  EA 49 00 00 FE      jmp far FE00:0049
```

If the image is instead burned into the *BIOS* socket, offset `1FF0` is physical `FFFF0` — the 8088 reset vector — and that jump lands on the same entry point. The code knows which situation it is in:

```
951:  mov ax,cs
      cmp ax,0xfe00
      jne 0x975            ; under a real BIOS: use INT 10h and read the BIOS data area
```

```
      ; otherwise there is no BIOS. Program the MDA ourselves:
      mov dx,0x3b8 ; mov al,1 ; out dx,al
      mov dx,0x3b4                        ; CRTC index/data at 3B4/3B5
      mov ax,0xb000 ; mov es,ax
      mov cx,0x10                         ; 16 CRTC registers from the table at 09DE
```

and it initialises the 8255 peripheral interface, the 8259 interrupt controller and the 8253 timer unconditionally. The same 8 KiB serves either as an add-in boot ROM under IBM's BIOS or as the entire firmware of a machine that has none.

5.2 Self-relocation

```
77:  mov ax,0xff69 ; sub ax,0xfe00 ; mov bx,cs ; add ax,bx ; mov ds,ax   ; DS = CS + 169h
83:  mov ax,0x1fa0 ; mov es,ax ; xor si,si ; xor di,di
8d:  mov cx,WORD PTR ds:0x30f       ; = 030Fh
92:  repnz movsb
94:  push es ; pop ds
```

It copies a 783-byte initialised data image from ROM offset `1690` to physical `1FA00`, then runs with `CS` pointing at the ROM — for code and message strings — and `DS` at that RAM shadow, for variables. This detail matters when reading the disassembly: every `ds:` reference is a *variable*, whose power-on value is the corresponding byte at ROM offset `1690`$+n$. Reading them as ROM code, as is tempting, gives nonsense.

5.3 Screen

Messages are a four-byte header — video offset, attribute, length — followed by text, blitted straight into video memory as character/attribute pairs. There is exactly one `INT 10h` in the whole ROM, used only to set the mode.

Offset	Row, col	Attribute	Text
013B	8, 31	07 normal	`VERSION:  2.20`
014D	10, 31	87 blinking	`... LOADING ...`
0160	10, 26	F0 reverse	`!!! NO NETWORK FOUND !!!`
017C	12, 26	07	`FROM LOCAL AREA NETWORK` ← the Destek card
0197	12, 31	07	`FROM IBM PC-NET`
01AA	12, 31	07	`FROM EAGLE USART`
01BE	12, 32	07	`FROM IBM UART`
0CEB	12, 32	07	`FROM TIGAN LAN`
089A	14, 32	0F bright	`ADDRESS:` + three decimal digits
01CF	16, 30	70 reverse	`!!! TIME OUT !!!`
01E3	16, 27	70	`!!! DUPLICATE ADDR !!!`
01FD	16, 26	70	`!!! LAN BOARD FAILED !!!`
0219	16, 28	70	`!!! CHECK SERVER !!!`
0231	16, 27	70	`!!! BAD INTERRUPT !!!`

Character 0 of the screen is incremented on every received block, giving a flickering progress indicator in the top-left corner.

5.4 Finding the card

```
281:  test BYTE PTR cs:0x6,0x40 ; je 0x28c ; xor ax,ax ; ret   ; ROM[6] bit 6 = skip this LAN
28c:  mov dx,0x220 ; out dx,al       ; a write to the even port resets the card
290:  mov dx,0x36e ; out dx,al
294:  mov bx,0x9080                  ; bl = 80h -> PIC mask bit for IRQ 7
297:  mov cx,0x2800 ; loop $         ; give the Z80 time to boot
29c:  mov cx,0x1000
29f:  in al,dx ; test al,0x80 ; loope $-3   ; wait for "the card has a byte for you"
2a6:  je 0x2b2                       ; timed out -> try the other address
2a8:  or dl,1 ; in al,dx             ; read the greeting byte
2ae:  cmp al,0x6f ; jbe 0x2c1
2b2:  mov ax,dx ; mov bl,8 ; mov dx,0x220 ; and al,0xfe ; cmp ax,0x220 ; jne 0x29c
2c0:  ret                            ; both addresses tried; nothing there
2c1:  or al,al ; jne 0x2f6           ; a non-zero greeting -> LAN BOARD FAILED
2c5:  or ds:0x5a,bl                  ; remember which IRQ to unmask
```

```
2d3:  mov si,0x17c ; call 0x9ee      ; print "FROM LOCAL AREA NETWORK"
2d9:  mov si,7 ; mov cx,0x13 ; push cs ; pop ds ; call 0xf99   ; send 19 config bytes
2e6:  and dl,0xfe ; in al,dx ; test al,0x80 ; je $-3            ; wait for the answer
2f1:  or dl,1 ; in al,dx ; or al,al ; jne 0x2f6                 ; non-zero -> LAN BOARD FAILED
2fc:  mov ds:0x1a,dx ; and dl,0xfe ; mov ds:0x18,dx             ; latch the port numbers
307:  mov ds:0x5b,8 ; mov ds:0x5c,0x14 ; mov ds:0x5d,3
316:  mov ds:0x1e,0x0f93 ; mov ds:0x20,0x0f90                   ; install the driver vectors
```

This meshes exactly with the card's own reset routine, which posts `00h` on power-up, swallows 19 bytes of configuration, and answers `00h` for success or `01h` for failure. It also pins down the card's addressing: `36Eh` implies IRQ 7, `220h` implies IRQ 3.

The DIP setting recorded in the uploader's notes — a nine-way switch, all on except 5 and 9 — decodes to `220h` if positions 1 through 9 correspond to address lines A1 through A9 with "on" meaning zero: only A5 and A9 high, giving $020 \mid 200 = 220$. That is precisely the second address MAGIX probes, and its `220h` branch selects IRQ 3, which is consistent with the two-way SW2. The inference is not proven from the traces, but it is well supported.

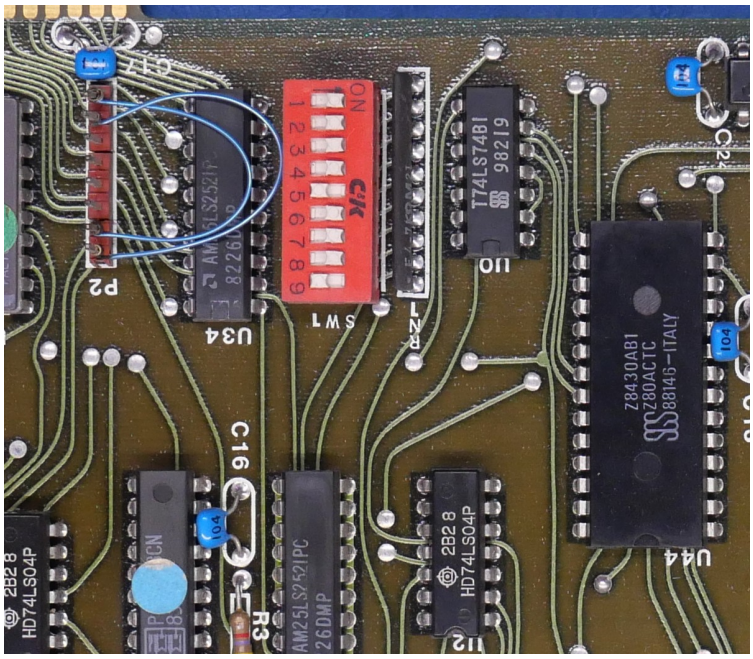


Figure 6: SW1, the nine-position address switch, with the Am25LS2521 comparator (U34) to its left and the P2 option header beyond that, carrying two blue strap wires. RN1, the pull-up network, is to the right.

5.5 The other four adapters, and what "9600" means

MAGIX is not a Destek product and does not assume a Destek card. It probes five adapters in a fixed order and installs a small vtable of near-pointers for whichever answers first; everything above that layer — the windowed block requester, the timer machine, the screen, the hand-off — is adapter-independent.

Probe	Screen	What it is
0281	FROM LOCAL AREA NETWORK	the Destek card; mailbox at 36E or 220
0B17	FROM TIGAN LAN	an Intel 82586 Ethernet board at 31E/31F with 32 KiB of shared memory at A8000
0325	FROM IBM PC-NET	the IBM PC Network adapter. It probes *no port at all*: it reads the segment of the INT 5Ch (NetBIOS) vector at 0000:0172 and requires CC00, the adapter's option-ROM segment
03EA	FROM EAGLE USART	a Zilog Z8530 SCC serial board at 300–304
04CD	FROM IBM UART	plain COM1, an 8250 or 16450 at 3F8

The three intelligent boards hand over whole packets and need only two vtable slots. The two serial cases share a driver and add a software CRC-16 — the table with the defective entry from §5.7 — plus a byte-stuffed

framing layer and two more slots.

That also explains the last unread thing on the EPROM label. **ROM offset 6 is a per-station option byte.** Bits 6, 5 and 2 individually inhibit the Destek, PC-NET and Tigan probes, and **bits 1–0 select the serial line speed**: 9600, 4800, 2400 or 1200 baud. The label reads `STA-36` and `9600` because those are the two things a technician had to set per machine — the station number at offset 7, and the line speed in the bottom two bits of offset 6.

5.6 Booting

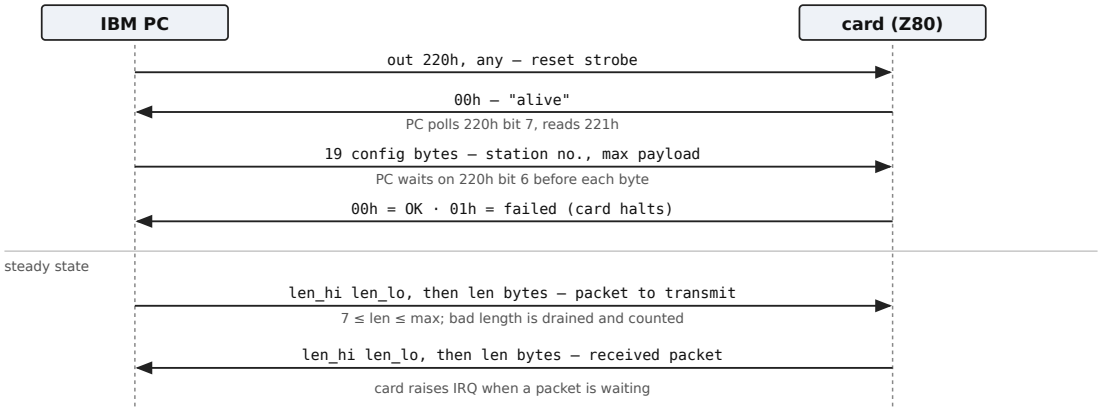


Figure 7: The complete host-to-card conversation, from reset to steady state. Both sides read the same two status flags; only the sense of "which direction is ready" differs.

Once a card is found, MAGIX asks a fixed server for a boot image. The destination in its packet header is initialised from the ROM data image to `FE FF 00` — so **the boot server is station `FFFE`**, whose SDLC address field is `FEh`. It is a well-known address, not a broadcast; the firmware's broadcast is `FFFF`. Hence the `!!! CHECK SERVER !!!` message when nothing answers.

MAGIX uses **packet type 0**, which is why it never collides with the card's own type 1 and type 2 management traffic.

The image arrives in 128-byte blocks. Block n is written to segment `0040` at offset $n \times 128$; blocks 0 and 1 are staged at segment `0800` first. Block 0 carries a header whose word at $+4$ is the total block count and whose word at $+6$ is the entry offset. The requester keeps up to eight blocks outstanding, with retransmission driven by the timer interrupt.

When the last block lands, the ROM leaves the station number in the first word of memory for the downloaded program to find, restores the interrupt controller and timer to IBM defaults, and jumps:

```
7c5:  xor bx,bx ; mov es,bx
7c9:  mov al,cs:0x7 ; mov ah,cs:0x6
7d2:  mov es:[bx],ax        ; station number + flags at 0000:0000
...
934:  jmp DWORD PTR ds:0x36 ; segment forced to 0040h
```

5.7 A forty-year-old bug, preserved

The ROM carries a 256-entry CRC lookup table at `ds:010F`, i.e. ROM offset `179F`. It is the standard CRC-16/ARC table — reflected, polynomial `A001` — stored as little-endian words. Every entry is correct except one:

Index	In the ROM	Correct
A4	01 B8	01 BB

The wrong byte is at ROM offset `18E8`. Two things say this is an original defect in the burned master rather than decay in a forty-year-old EPROM:

- **The bits went the wrong way.** `B8` is `10111000`; `BB` is `10111011`. The ROM has *fewer* set bits than it should. An EPROM cell is erased to 1 and programmed to 0, and what decades of charge leakage does is turn programmed `0`s back into `1`s. Rot adds ones; this byte is missing two.
- **The checksum still passes.** The image's 8-bit sum is exactly `00`, which is what the BIOS requires before it will execute an option ROM. A byte that changed after programming would have broken that.

So the table was wrong when it was burned, and every workstation that got this revision got the same wrong byte. Whether anyone ever noticed depends on which network they were on: the Destek card computes its frame check sequence in the WD1933, in hardware, and never touches this table. It is the byte-serial transports — the ones running over a UART, where the CRC has to be done in software — that would have felt it. On a Destek network the bug is invisible, which is a fair guess at how it survived into production.

6 What this was for

Put the two ROMs together and the product is unambiguous. A small office buys one machine with a hard disk and calls it the server. Every other desk gets a plain IBM PC, a Destek card, a drop cable, and a boot EPROM burned with its own station number. Switch a workstation on and it finds its card, announces itself, pulls its operating environment down the wire in 128-byte pieces, and runs — with no local disk involved in the decision.

That was a real and pressing problem in 1983. A 10 MB hard disk cost more than the PC attached to it. Diskless workstations on a cheap twisted-pair LAN were how you gave a dozen people a computer each without buying a dozen hard disks, and a fleet of nearly identical products existed to serve that market: Corvus Omninet, Nestar Cluster/One, Sytek LocalNet, Orchid PCnet, 3Com's EtherLink, and from 1984, IBM's own PC Network.

The card turns out to belong to exactly that company, and not by inference. Ordinary web searching returns nothing whatever for "The Destek Group", but the Internet Archive's scanned trade literature has them in several directories: founded 1981, at 830 East Evelyn Avenue, Sunnyvale, California, selling a local area network called **DESNET**. One of those books, *The Local Area Network Book*, carries a chapter on "some real products" comparing **Corvus Omninet, Desnet, and Ethernet components by 3Com** — placing this card, from the literature rather than from guesswork, in precisely the company the silicon suggested. Appendix E has the details, and the negatives.

Two things stand out from the silicon alone. First, the date: a 1982 board doing CSMA/CD in firmware is contemporary with the very first commercial Ethernet cards. Second, the autonomy: a station-status protocol answered by the adapter's own processor, so a machine that has crashed — or has not even booted — still shows up on a network scan with a full set of error counters. That is a diagnostic capability that did not become ordinary until much later.

What the silicon does *not* support is speed. The WD1933-03's own timing table caps this card at 78 kbit/s in the clock mode a multidrop bus needs (Appendix B). DESNET was not competing with Ethernet on throughput. It was competing on the price of a hard disk.

The card in the archive was pulled from a working IBM PC 5150 whose motherboard still carried the matching boot EPROM, labelled by hand in February 1986. Somebody used this, in earnest, for at least four years.

A Protocol specification

This appendix is written to be implementable. Everything in it is derived from the two ROM images; where a statement is an inference rather than a direct reading, it says so.

Throughout, *station address* means the three-byte field carried in a packet, and *SDLC address* means the single byte the WD1933 matches in hardware — which is the first byte of the station address.

A.1 Physical and framing layer

Standard HDLC/SDLC, produced and consumed entirely by the WD1933B-03:

- Frames are delimited by FLAG octets, `7E` (`01111110`).
- Zero-insertion and deletion are automatic: a `0` is inserted after five contiguous `1` bits on transmit and removed on receive.
- The frame check sequence is a 16-bit CRC with the standard HDLC polynomial $x^{16}+x^{12}+x^5+1$, generated and verified in hardware. The residue on a good frame is `F0B8`.
- Seven or more contiguous `1` bits is an ABORT; fifteen or more is IDLE.
- AUTO FLAG is enabled, so the line carries continuous flags between frames rather than going idle.
- Loop mode is *off* — this is a contention bus, not a ring.
- Characters are eight bits; there is no extended address or control field and no residual character.

The line rate is set on the card by dividing the 32.000 MHz oscillator; see Appendix E.

A.2 Addressing

Address	SDLC A-field	Meaning
`FFFF`	`FF`	broadcast. `FF` is the WD1933's hardware *global address*, so every station's receiver accepts it without any extra configuration.
`FFFE`	`FE`	the MAGIX boot server. A well-known unicast address, not a broadcast.
`0000`	—	rejected at configuration time; means "no address supplied".
other	low byte	a station. This card was configured as `0036`.

A receiver applies two filters. The WD1933 compares the A-field against its address register and raises a data request only on a match or on the global address; the firmware then compares the full 16-bit destination (A-field as the low byte, C-field as the high byte) against its own and discards anything that is neither its own address nor `FFFF`.

The third destination byte and the third source byte are carried but never examined by the card firmware. MAGIX sets both to `00`. They are available for a higher layer.

A.3 Packet format

A packet, as counted by the length field and as laid out from buffer offset $+8$:

Offset	Field	Notes
0	destination, low	becomes the SDLC A-field; hardware-matched
1	destination, high	becomes the SDLC C-field
2	destination, third byte	carried, not interpreted
3	source, low	**overwritten by the sending card with its own address**
4	source, high	likewise
5	source, third byte	supplied by the host, carried unchanged
6	type	`00` MAGIX `01` status request `02` status reply
7…	payload	

Minimum packet length is 7 — the header alone. The maximum is $4P + 7$, where P is the size parameter in the configuration block, so the *payload* maximum is exactly $4P$. MAGIX sets $P = 34$, giving 136 bytes of payload in a 143-byte packet; the card's own ROM default is $P = 64$, giving 256 and 263.

The bound is easy to misread, and worth spelling out because it is the number a server implementer needs. The check in SUB01 is two subtractions with no reload in between:

```
010B:  ld bc,0007h
       xor a
010F:  sbc hl,bc        ; HL = len - 7 ... and HL is NOT restored
0111:  jp c,0179h       ; len < 7            -> reject
0114:  ld bc,(2057h)    ; BC = 4P
0118:  inc bc           ; BC = 4P + 1
011A:  sbc hl,bc        ; HL = (len - 7) - (4P + 1)
011C:  jp nc,0179h      ; len - 7 >= 4P + 1  -> reject
```

So the second comparison is against $len - 7$, not len. Two independent facts confirm the result. The receive buffer's DMA area is $stride - 8 = 4P + 9$ bytes, which is exactly $(4P + 7) + 2$: the largest legal packet plus its two FCS bytes, and not one byte more. And $4P$ is the round number — 136 for MAGIX, 256 for the ROM default — which is what a size parameter is for.

A source address cannot be forged from the host: the card overwrites bytes 3 and 4 of every outgoing packet with its own configured address.

Buffer layout in card RAM

Useful when reading a memory dump of a live card. Each buffer is $4P + 17$ bytes.

Offset	Field	Written by
$+0, +1$	link to next buffer	queue routines
$+2$	class: 00 receive ring, 01 free pool	buffer-pool setup
$+3$	WD1933 interrupt register at frame end	the ISR
$+4$	WD1933 status register at frame end	the ISR
$+5$	FF once the DMA has filled it	the ISR
$+6, +7$	length, big-endian	the ISR (raw residual), then the delivery routine (true length)
$+8 \ldots$	the packet	DMA
after the packet	two FCS bytes	DMA; excluded from the reported length

The 8237's channel-2 count register is loaded with $stride - 9 = 4P + 8$, and the 8237 transfers $count + 1$ bytes, so the DMA area is $4P + 9$ bytes running from $+8$ to the end of the buffer. The delivered length is computed as

$$len \;=\; (\,stride - 11\,) \;-\; residual$$

where *residual* is read back from the 8237's channel-2 count register. Since the DMA count was $stride - 9$ and the 8237 is loaded with $n - 1$ for n transfers, the bytes actually moved are $(stride - 9) - residual$: the firmware subtracts a further two. The controller DMAs the two FCS bytes into memory along with the frame, and the firmware excludes them from the reported length — which is why the DMA area is sized at exactly $4P + 9$, the largest legal packet plus its CRC — so the CRC of every received frame is sitting in RAM immediately after the payload, should anyone want it.

A.4 Station-status protocol

This is the part of the card most worth exploiting today, because it works with no software running on the host at all.

Request. Any packet of type 01, addressed to a station or broadcast. The card examines only the type byte; the payload is ignored. A minimal request is therefore seven bytes:

```
dest_lo dest_hi dest_x  src_lo src_hi src_x  01
```

Reply. The card builds a 37-byte packet of type 02, addressed to the requester's source address, with its own address as the source and the third source byte forced to 00. The 30-byte payload is:

Payload offset	Width	Contents
0–5	6 bytes	the ASCII string `mdn100`, copied from ROM `00A5`
6–9	32 bits	frames transmitted successfully
10–13	32 bits	frames received and processed
14–17	32 bits	frames received but addressed elsewhere
18–19	16 bits	CRC errors
20–21	16 bits	overruns and bad residual lengths
22–23	16 bits	aborted or invalid frames
24–25	16 bits	transmit retries after collision or underrun
26–27	16 bits	receive overruns — the ring had no free buffer
28–29	16 bits	packets rejected from the host for a bad length

All counters saturate rather than wrap: the increment routine backs the value out again on overflow, so a 16-bit counter sticks at `FFFF` and a 32-bit one at `FFFFFFFF`. They are stored least-significant byte first at the lowest address, incremented from the low byte upward.

The card answers a type-1 request from its own host as well, looping the reply back up the mailbox rather than putting it on the wire.

A.5 Host interface

Two ISA I/O ports, at base `220h` on this card (`36Eh` is the alternative). The status port is the even one; the data port is the odd one. Both sides read the same two flags.

Operation	Port	Effect
write anything	`220`	resets the card's Z80
read	`220`	bit 7: a byte is waiting for the host. bit 6: the card can accept a byte. Other bits not used by either ROM.
read	`221`	take the waiting byte
write	`221`	hand a byte to the card

Both directions are byte-at-a-time programmed I/O with a full handshake on every byte; there is no block transfer and no host-side DMA. Both directions frame a packet identically, as a two-byte big-endian length followed by that many packet bytes.

Reset and configuration

1. Write any value to `220`. The Z80 restarts.
2. Wait — MAGIX allows roughly 10 000 dummy loop iterations — then poll `220` until bit 7 is set and read `221`. The card posts `00` to say it is alive. Anything else means the card is faulty.
3. Send the 19-byte configuration block, waiting for bit 6 of `220` before each byte.
4. Poll for bit 7 and read the answer: `00` means configured and running; `01` means the card rejected the configuration and has executed `halt`. Only a further write to `220` will revive it.

Sending a packet

1. Wait for bit 6 of `220`; write the high byte of the packet length to `221`.
2. Wait for bit 6; write the low byte.
3. Repeat for each packet byte. Length must satisfy $7 \le len \le 4P + 7$.

A length outside that range is not an error the host is told about: the card silently reads and discards that many bytes, increments its "bad length" counter, and carries on. There is no acknowledgement that a packet was accepted, transmitted, or lost — if the host needs to know, it must poll the card's own statistics with a type-1 request.

Receiving a packet

1. Wait for bit 7 of `220`; read the high byte of the length from `221`.

2. Wait for bit 7; read the low byte.
3. Read that many packet bytes the same way.

Interrupts

The card asserts its ISA interrupt line — IRQ 3 at base 220h, IRQ 7 at base 36Eh — when it has data for the host. MAGIX proves this: it installs the *same* handler at both IRQ 3 (0000:008C) and IRQ 7 (0000:009C), and that handler does nothing but drain the mailbox:

```
5E6:  push ax
      call 61Dh          ; shared prologue: pop the return address into AX and call it back
5EA:  mov  dx,ds:18h     ; the card's status port
      in   al,dx
      test al,80h        ; a byte waiting?
      je   5C6h          ; no - return
      call 0F96h         ; yes - read one whole packet
      jmp  5EAh          ; and keep going until the card is drained
```

The four-byte prologue at 61D (`pop ax / call ax / pop ax / iret`) is shared by all six of the ROM's interrupt handlers: `pop ax` takes the return address the `call` just pushed, `call ax` re-enters the handler body, and the second `pop` restores the caller's AX before the `iret`.

Separately, the card writes to port 60h — a write-only strobe with no readable side — at the moment it becomes ready to *accept* a packet from the host. That is a different event from "a packet has arrived", and the two ROMs do not settle whether 60h drives the same interrupt line, a second one, or something else entirely; see Appendix E.

A host that does not want interrupts can simply poll the status port. MAGIX itself does both: the interrupt handler drains the card, and the main loop polls as well.

A.6 The configuration block

Nineteen bytes, sent by the host immediately after reset. The card stores them at 2040–2052 and then substitutes ROM defaults for any of the last ten that are zero.

Byte	Field	Default	MAGIX	Meaning
0–1	station address	—	36 00	16-bit, little-endian. If zero, the card reads its DIP switches instead. 0000 and FFFF are rejected.
2–8	unused	—	zero	not read by this firmware
9	reserved	05	FF	stored, never read
10	reserved	00	00	stored, never read
11	size parameter P	40	22	must be ≥ 8. Sets maximum payload $= 4P$, maximum packet $= 4P + 7$, buffer stride $= 4P + 17$.
12–18	unused	00	00	

With MAGIX's $P = 34$ the buffer stride is 153 bytes; the card's 4 KiB of RAM then yields 25 buffers — 20 in the receive ring and 5 held back as a free pool — occupying 2097–2FF7 and leaving 120 bytes unused.

Failure codes

01 is returned for a bad station address, and — because of the dead store documented in §4.7 — also for a size parameter below 8, which was meant to return 07. Either way the card halts and must be reset.

B Building the server side today

The card is one half of a conversation whose other half no longer exists. This appendix is about supplying it. Two facts dominate everything else, and both are recent findings rather than assumptions carried over from the obvious reading.

B.1 The line is an IBM System/360 party line, not a serial link

U1 and U19 are a DS75123 dual line driver and a DS75124 triple line receiver. Both Texas Instruments datasheets (committed alongside this report) open with the same sentence: *meets or exceeds the requirements of the IBM System/360 input/output interface specification*. The driver's output stage is described as an *uncommitted emitter-follower output structure for party-line operation*. Its V_{OH} is 3.11 V typical at -59.3 mA, and its V_{OL} is specified only at $-240\,\mu$A — the output physically cannot sink current.

That settles the electrical layer:

- **Single-ended and ground-referenced**, not differential. It is not RS-422 and it is not RS-485.
- **Idle low, asserted high** at about 3.1 V, into a resistive termination to ground.
- **Wired-OR with high dominant** — any station asserting pulls the whole bus high.
- Receiver threshold around 1.2 V with 0.2–0.5 V of hysteresis (the DS75124 guarantees $V_{IH} \geq 1.7$ V, $V_{IL} \leq 0.7$ V).

This is dominant/recessive arbitration — structurally the same trick CAN uses, seven years early — and it is almost certainly *how* Destek got collision detection so cheaply. Transmit a recessive bit, read back a dominant one, and you have lost the line. The collision signal the firmware reads as SR bit 7 costs one comparator.

An RS-485 transceiver will not work. It is differential, its output is push-pull so it cannot wired-OR, and its common-mode range assumes a balanced pair. Converting requires an active bridge — RS-485 transceiver, comparator, emitter follower — with the arbitration re-implemented behind it. Building the real thing is easier.

The single luckiest fact in this whole exercise: TI still lists both the **SN75123** and **SN75124** as active, orderable parts. A 1973 device is still in production, which removes essentially all of the electrical guesswork. Verify distributor stock before designing around it. If you must build a discrete equivalent, the driver's internal schematic is on page 3 of the committed datasheet: an NPN emitter follower with roughly 360 Ω of base drive and 15 Ω in series with the emitter. Gate the transmit data with the request-to-send signal; an ungated idle transmitter emits continuous ones and would jam a dominant-high bus permanently. For the receive side a 74HCT14 at 5 V is a surprisingly good threshold match, or use a real comparator against a 1.2 V reference.

B.2 The line rate is far lower than it looks

The card carries a 32.000 MHz oscillator, which invites the conclusion that the network runs at 32 MHz$/32 =$ 1 Mbit/s in the WD1933's 32× clock mode. **That is wrong**, and this report said it before the datasheet's AC table was checked properly.

Table 4 on page 112 caps the *external clock* per speed grade, and U15 is a WD1933B-**03**:

Grade	32× max	1× max
WD1933-00/10	2.0 MHz	0.5 MHz
WD1933-01/11	2.0 MHz	1.0 MHz
WD1933-02/12	2.0 MHz	1.5 MHz
WD1933-03	**2.5 MHz**	**2.0 MHz**

32 MHz is 12.8 times over the 32× limit for every grade in the family. The oscillator is the *board* clock: 32 MHz$/8 = 4$ MHz is exactly a Z80A and an 8237A-**4**, which is what one of the two 74LS93s is for. The second can only hand the WD1933 2 MHz, 500 kHz or 250 kHz.

So the ceilings are **78.125 kbit/s** in 32× mode and 2.0 Mbit/s in 1× mode, and the most likely actual rate is **2 MHz in 32× mode = 62.5 kbit/s**. The argument for 32× is that it is the only mode in which the chip's digital phase-locked loop runs, and a multidrop bus with no clock wire has to recover its clock from the data. The counter-argument is that the DS75124 is a *triple* receiver and the firmware only uses three inputs' worth of information, so a distributed clock on a fourth line cannot be ruled out from the photographs alone.

Both the encoding (NRZI, pin 28) and the clock mode (pin 30) are hardware straps. They are invisible to the firmware, so no amount of further disassembly will settle this. It is a continuity meter and a frequency counter, ten minutes' work, on a card in hand.

At 62.5 kbit/s the whole problem changes character. A full eight-frame window is 8×143 bytes $= 146$ ms on the wire, against a client retransmit timer of 199.5 ms — comfortable, but not the three-orders-of-magnitude margin a megabit line would give. And the host PC stops being the bottleneck: MAGIX's mailbox loop costs roughly 18 μs per byte on a 4.77 MHz 8088, so draining a 141-byte packet takes about 2.6 ms, well inside the 18 ms that packet spends on the wire.

B.3 What to measure first

In this order, before connecting anything to anything:

1. **Card unpowered, continuity meter.** U15 pin 30 (1X/32X) and pin 28 (NRZI), each to ground and to V_{CC}. Low means 32× and NRZI. This one check settles the encoding and the rate class.
2. **Measure R6 and R7**, between J3 and U1/U19. This tells you whether the card terminates the bus or merely biases it, and therefore whether you may add a terminator.
3. **Continuity from U1 pins 7 and 9** (driver outputs) and **U19 pins 14, 3 and 10** (receiver inputs) to the fifteen pins of J3, and find the ground. *This is the pinout*, and it is not recoverable from photographs at any resolution.
4. **Powered, no cable, ×10 probe to ground**, one pin at a time. Probing differentially will fool you into thinking it is RS-422. Expect idle near 0 V and about 3.1 V asserted.
5. **Frequency counter on U15 pin 31 (TC) and pin 26 (RC).** The bit rate is that value, or that value over 32 in 32× mode.
6. **Scope a frame.** Look for a separate 50 %-duty clock on another DB-15 pin: present means 1× mode with a distributed clock; absent means 32× with the DPLL. That is the discriminating measurement.
7. Expect **continuous flags** between frames, not an idle line — the firmware sets AUTO FLAG. A receiver that assumes mark-idle will mis-frame.
8. **Bond the chassis grounds.** A ground-referenced bus with half a volt of hysteresis has no tolerance for a ground offset.

B.4 Choosing a transceiver

Route	Verdict
RP2040 / RP2350 (Pi Pico)	**Recommended.** Two PIO state machines — one encoding NRZI with zero-insertion, flags and FCS, one hunting flags and undoing both — with the FCS in C on the main core. At 62.5 kbit/s this is not remotely tight. Carrier sense falls out for free ("has the receive state machine seen a transition in the last N bit times?"). Critically, it is the only route that can abort a transmission within a bit time of losing arbitration.
Linux generic HDLC + a sync serial card	Works for framing, but **cannot make a well-behaved CSMA station**: the kernel HDLC stack has no notion of "do not transmit while carrier is present" and no path to abort mid-frame. Also, the drivers have been dying — `z85230`/`sealevel` and Lanmedia were removed in Linux 5.19 (2022), `cosa` likewise, `dscc4` in 2019. What survives in-tree includes `farsync` (FarSite, PCIe and USB), `n2`, `c101`, `pci200syn`, `wanxl`. FarSite still sells hardware and ships driver source.
SyncLink (MicroGate)	**Do not build on this.** `synclink` and `synclinkmp` were removed in Linux 5.11 (2020); `synclink_gt` was marked broken in 2023 and removed in 2026, the commit noting "no evidence of users since 2016".
FPGA with an HDLC core	Viable but heavier than the problem deserves. CAST's HSDLC is the only actively supported commercial core; the OpenCores HDLC controller has not been touched since 2014 and its own page says it needs more verification.
Second Destek card	The most period-accurate answer, and it needs no PHY work at all — but you need a second card, a second host, and the drop cable that no longer exists.

The unavoidable conclusion is that a $4 microcontroller beats every PC synchronous serial card here, because the hard requirement is not the framing — it is reacting inside a bit time, which no USB- or PCIe-attached host stack can do.

B.5 The server software

Put the abstraction boundary at **whole packets**, not bits: everything above it is transport-independent, and every PHY differs only in whether it does framing and FCS itself.

```
phy/base.py        recv() -> bytes|None ; send(packet) ; carrier() -> bool
phy/serial_mcu.py  COBS over USB-CDC to the Pico doing real HDLC
phy/linux_hdlc.py  AF_PACKET SOCK_RAW on an hdlc0 from ‘sethdlc hdlc0 raw‘
phy/loopback.py    in-process, for tests
phy/pcapfile.py    replay and record
destek.py          packet parse/build, destination filter, type dispatch
magix.py           the boot session state machine
mgmt.py            type 1 -> type 2
image.py           block-0 header synthesis and slicing
server.py          one reader, no timers except session garbage collection
```

The kernel's `hdlc_raw` protocol is a particularly good fit: it strips flags and the FCS and hands you exactly the address, control and information fields — which *is* the Destek packet, verbatim.

If a PHY needs the FCS in software it is CRC-16/X-25: polynomial `1021` reflected to `8408`, initial value `FFFF`, final XOR `FFFF`, least-significant bit first. Check it against `906E` over the string `123456789`, and expect the receiver residue `F0B8` that the WD1933 datasheet quotes.

The state machine

The server is **purely reactive**. The client owns every retransmission timer, so the server needs no timers at all beyond garbage-collecting idle sessions. Keyed on the client's 16-bit source address:

Received	Name	Response
cmd 4	connect	reset the session; reply cmd 5 with ack = the client's seq and credit = seq + 8
cmd 0	request	parse the body; send the blocks with seq = client's ack + 1 upward, ack = the client's seq
cmd 3	NAK	$(ack - base) \bmod 256$ blocks were accepted; resend from there
cmd 1	ACK	nothing
cmd 2	window stall	send a fresh credit
cmd 6	done	drop the session

Three rules make it work, and all three are load-bearing:

1. **Never keep an independent sequence counter.** The client accepts a data packet only if its sequence is exactly its own last-accepted plus one. So the server's next sequence is always *the ack byte of the client's most recent packet*, plus one, recomputed every time. Idempotence then falls out for free: a retransmitted request carries the same numbers and reproduces exactly the right resend.
2. **The ack byte must echo the client's request sequence.** That, and nothing else, clears the client's 199.5 ms retransmit timer.
3. **Credit is a signed 8-bit comparison.** Advertise client sequence $+\,8$; never let the window exceed 127.

Two ways to make the workstation reboot instead of boot

Both are worth knowing before the first test, because both look like hardware faults:

- **Never send a data packet with an empty body.** The client falls straight through to `!!! LAN BOARD FAILED !!!`.
- **Never send a packet of any type other than 0 to a station that is booting.** The same fatal path. In particular, if the server answers a broadcast type-1 poll by broadcasting its reply, it will kill every workstation currently loading. Reply *unicast*, always — which is exactly what the card itself does.

Timing budget

Every number below comes from the 8253 channel-0 reload of `7436` at 1.193182 MHz, i.e. a 24.933 ms tick, rearmed by the timer interrupt on every expiry.

Timer	Period	Meaning
request retransmit	8 ticks, 199.5 ms	a duplicate request arrives if the first data packet is later than this
connect / stall retry	20 ticks, 498.7 ms	
post-block idle	3 ticks, 74.8 ms	
no-progress budget	10 expiries	→ `!!! TIME OUT !!!`; 1.99 s while requesting, 4.99 s while connecting
absolute deadline	600 ticks, 14.96 s	rearmed on every block placed

Testing without hardware

The protocol is fully specified above, so a loopback PHY plus a client simulator written from the ROM exercises everything except the wire. Beyond that, the card is a realistic candidate for emulation in MAME: `bus/isa/3c505.cpp` and `bus/isa/np600.cpp` are already ISA cards carrying their own CPU and their own network controller, running the card's real firmware. Every part the Destek needs exists — `z80`, `z80ctc`, `am9517a` — *except* the WD1933. MAME's `machine/sdlc.cpp` gives bit-level flag detection, zero-deletion and FCS accumulation, but it is receive-only and there is no register-level SDLC controller anywhere in the tree. A `wd1933_device` is perhaps 600–900 lines: six CPU-visible registers, the data requests wired to the DMA controller, the interrupt request wired into CTC channel 0, the 8-bit address comparator with its global address, auto-flag, and a bit-serial transmit/receive pair. Then a small "wire" device to join two instances, and two emulated 5150s can boot each other.

C Things worth demonstrating

Everything here is bounded by what the hardware and firmware actually do. Four properties make this card unusually rewarding to play with:

- **The card is an autonomous peer.** It answers type-1 status requests entirely from its own Z80, never tells the host, and keeps running whether or not the 8088 is alive.
- **The host interface is trivial** — a two-byte length and then the packet, identically in both directions. A PC-side program is about a hundred lines of assembler, or a session in `DEBUG`.
- **The whole boot protocol is now readable**, byte for byte.
- **The frame budget is host-selectable.** A program that resets the card and sends its own 19-byte configuration block can raise the limit from MAGIX's 136-byte payload to the ROM default of 256.

Ranked

1. Build the missing transceiver. Nothing analogue exists on the card: J3 goes through two resistors to the driver and receiver and nothing else, so the media attachment lived in a box on the drop cable that nobody has. A Pico with the two PIO state machines and a DS75123/DS75124 pair wired as the mirror image of U1 and U19 is the prerequisite for everything below that touches the wire. One wiring detail matters more than the rest: your peer must assert the card's carrier line *only* while actually transmitting. Leave it asserted and the card's carrier sense will see a permanently busy line and it will never speak at all.

2. Loopback self-test, drop cable unplugged. Needs no electronics whatever. From `DEBUG` on the 5150: reset the card, feed it 19 configuration bytes, then send it a packet addressed to *its own* station number. The firmware compares the destination against its configured address and, when they match, queues the packet straight back to the host instead of to the transmitter. Make the type byte 1 and the card replaces your packet in place with its own 37-byte status reply — the string `mdn100` and nine live counters. That is the entire card proved out, on the bench, with a working machine and no network.

3. A network scanner that finds machines with no operating system. Put one seven-byte frame on the wire: `FF FF 00`, your address, type `01`. The hardware global address makes every card accept it; every card builds and returns a status reply; and not one of their host PCs ever learns it happened. Sweep it repeatedly and you have a live station map with error counters, covering machines that are powered but unbooted, or crashed, or have no software at all. This is the single most useful thing the card can do for a preservation project, and it works before you have written a single line of boot server.

4. Network-boot a real 5150 from a Raspberry Pi. The headline demonstration. Everything needed is in Appendix B; the first packet the workstation sends is known byte-for-byte, and a Pi answers three orders of magnitude inside every deadline.

5. Serve the boot menu. Nearly free once (4) works, and it is the part of the system nobody would guess exists. Answer the first request with a two-block "menu" whose first word is zero, followed by eight permission bytes and eight thirty-character labels. MAGIX saves the screen, paints `F1:` through `F8:` in reverse video, and polls the keyboard directly. A key is accepted only if that entry's permission byte, ANDed with a byte burned into *that workstation's own EPROM*, is non-zero. Per-station access control, in 1985, enforced by a byte in a ROM on the machine being controlled.

6. Boot something written this decade. The load environment is completely specified: your code lands at `0040:0100` upward, is entered with the BIOS's own stack, has the video, timer, interrupt controller and keyboard interface already initialised, and finds its station number in the first word of memory. There is a 64 KiB ceiling and no DOS. A demo, a text-mode chat client that talks to the other workstations over packet types 3 and up, or a diagnostic that has never existed before — all of it is a plain flat binary.

7. A live link-quality display. Poll one station several times a second and plot the nine counters. CRC errors, aborts, retries and ring overruns, updating in real time, from a card that is not running any software.

8. Provoke collisions and watch the backoff work. Two experiments that the firmware treats completely differently. Hold the carrier line asserted and the card never transmits at all — carrier sense, visible. Assert the collision line during a transmission and watch the retry counter climb by exactly five, then watch the packet be silently discarded while the "transmitted successfully" counter increments anyway. That last detail is a genuine defect worth demonstrating: after five failures the host is never told the packet did not go.

9. Capture the original boot traffic. Before writing any server, sit the transceiver on the drop cable in receive-only mode and record everything a real workstation says. It is a preservation artefact in its own right, and it settles by observation several things this report had to infer.

10. Reconfigure the card from DEBUG. The card reads its *entire* identity from the host at reset. Rename it, resize its frames, give it a different station number — or configure it as `FFFE` and make the 5150 itself the boot server for another 5150.

11. Bridge it to the modern world. Packet types 3 through 255 are entirely unassigned and the card carries them transparently. A gateway on types 3+ gives the 5150 a route off the segment without disturbing anything MAGIX does.

12. Prove that source addresses cannot be forged. The card overwrites the source field of every outgoing packet with its own configured address, in firmware, before transmission. A host program simply cannot lie about who it is — unless it resets the card and re-declares itself, which is exactly what (10) does. A nice, small demonstration of where a trust boundary really sits.

D Method, and how to check this

Everything in this report is reproducible from the repository that accompanies it. This appendix records how, because a claim whose evidence exists only in someone's terminal is a claim nobody can check.

D.1 Provenance

The source material is Internet Archive item `destek-isa-network-card`, uploaded 2025-04-24. `tools/fetch-ia-item.sh` re-downloads every original file and verifies each against the MD5 recorded in `ia/metadata.json`, which is archive.org's own manifest. The two ROM images are:

File	SHA-1	Size
`Destek MON100.BIN`	`832067abd0371d66669f20e6a8ef5c3c88637ddf`	4096
`5150 Option ROM.bin`	`1a188ac5e7e624617a91515d45cd45c7b0c8ef7a`	8192
`Card ROM.asm`	`3c0316e0b4685d8ab9d101693efa6c42539cb58b`	144124

D.2 Tools

Tool	Used for
MAME `unidasm`	Z80 disassembly, including the interrupt handler the supplied listing missed. Note that with `-skip` it prints addresses *relative to the skip point*.
GNU `objdump`	8086 disassembly (`-b binary -m i8086 -M intel`). It decodes linearly, so it mis-frames after every data block; regions were re-anchored with `--start-address` wherever the output looked wrong.
Python + PIL	cropping and upscaling the board photographs to read part markings.
`pdflatex`, headless Chromium	this document and its vector figures.

`tools/disassemble.sh` regenerates every listing in `disasm/`. `tools/svg2pdf.py` regenerates the vector figures from `figures/diagrams.html`. `report/Makefile` builds this PDF.

D.3 Verification

Findings were re-derived independently rather than accepted once. Each region of both ROMs was analysed, and then the resulting claims were handed to reviewers whose instructions were to *refute* them — one working from the raw bytes, disassembling the cited addresses again without trusting the cited evidence, and one working from the datasheets and the IBM PC 5150 programming model, hunting specifically for off-by-one, byte-order and bit-numbering errors. Claims that survived both are stated plainly here; claims that did not are either corrected or recorded in Appendix E.

It earned its keep. The clearest case is the maximum length, which this report got wrong twice before getting it right. The first reading took $4P$ from the name of the variable holding it. The second reading — a correction — re-derived it from the comparison at `0114` and concluded that $4P$ bounded the whole packet rather than the payload, because the comparison plainly reads `sbc hl,bc` against `(2057h)+1`. That was worse than the first: it missed that `HL` still held $len - 7$ from the *previous* subtraction eleven bytes earlier, which was never undone. A reviewer re-deriving the arithmetic from the raw bytes caught it, and the buffer geometry then confirmed the answer independently — the DMA area is exactly the largest legal packet plus its CRC. The same process is why the apparent bug in `SUB09` is documented here as *not* a bug.

D.4 Repository layout

`ia/`	the Internet Archive item, verbatim, plus its manifest
`documents/`	the WD1933 datasheet, without which the SDLC register writes cannot be decoded
`disasm/`	generated listings
`notes/`	the working analysis these appendices are drawn from
`figures/`	photograph crops and the SVG source for the diagrams
`report/`	this document
`tools/`	fetch, disassemble, figure conversion

D.5 What was *not* done

No hardware was involved. The card was not powered, no signal was measured, and no claim in this report has been confirmed against a physical device. Everything electrical — the line rate, the DB-15 pinout, what the collision detector actually senses — is inferred from firmware behaviour, datasheets and photographs, and is flagged as such where it appears. The first person to put a scope on one of these cards will settle several questions in an afternoon that no amount of disassembly can.

E Open questions, and what is known about the company

E.1 Questions the ROMs cannot answer

1. **The line rate.** Bounded, not determined: at most 78.125 kbit/s in 32× mode, at most 2 Mbit/s in 1× mode, most probably 62.5 kbit/s. The clock-mode and NRZI straps are hardware pins that the firmware never reads. Settled by a continuity meter on U15 pins 28 and 30 and a frequency counter on pins 31 and 26.

2. **The DB-15 pinout.** Two pins can be driven and three sensed. Which ones, and where ground is, requires a meter. No photograph will give it.
3. **What the collision detector physically senses.** The dominant-high wired-OR bus makes read-back-while-transmitting the obvious mechanism, but the comparator that does it is not on this board — it is in the missing media attachment box.
4. **Port `60h`.** A write-only strobe, fired when the card becomes ready to accept a packet from the host. Almost certainly the ISA interrupt request, but the card asserts an interrupt when it has data *for* the host too, and nothing in either ROM says whether these are the same line.
5. **SW1's second job.** It decodes the ISA address through the Am25LS2521. Whether the same switches are also what the Z80 reads at port `80h` for its fallback station number is unresolved.
6. **Configuration bytes 2–8, 9, 10 and 12–18.** Stored by the firmware and never read. Either a later revision used them, or the block is shared with another Destek product.
7. **The third address byte.** Carried in every packet, in both the source and the destination, and never examined. A subaddress or socket number for a layer above.
8. **What the boot image actually was.** The protocol downloads a flat binary and jumps into it. Nothing in either ROM constrains what that binary did.

E.2 Parts of the ROMs still undecoded

Stated so that the next person does not have to rediscover the gaps. The card firmware (`0000`–`0701`) is essentially fully decoded. In MAGIX:

- `0DFC`–`0ECA`, the ten state bodies of the byte-serial transmit machine, dispatched through a jump table at `0D1A`. Relevant only to the UART-based transports.
- `ds:0086`–`009A` and `ds:00D8`–`00F7` — 53 bytes of the RAM shadow reached only through register pointers. Related to the PC-NET adapter, which appears to use a buffer at physical `1F600`.

E.3 The Destek Group

Ordinary web searching returns nothing at all for either company. Full-text searching the Internet Archive's scanned-book corpus — the digitised trade literature of the period — returns a great deal. `tools/history-search/` reproduces these queries.

Company	The Destek Group, Inc.
Founded	1981
Address	830 East Evelyn Avenue, Sunnyvale, CA 94086; later Mountain View, CA
Telephone	(408) 737-7211
Product	**DESNET** — "a multipurpose LAN connecting mainframes and PCs"
Topology	listed as "Bus; logical ring (for fibre optics only)"
Also sold	**desNet BIOS**, compatibility software

Sources: *The Local Area Networking Sourcebook* (several editions), *Guide to Local Area Networks* (Byers), *The Local Area Network Book*, and an engineering software directory that names Dennis Rieger as marketing manager. *The Local Area Network Book* devotes a chapter to "some real products" comparing **Corvus Omninet, Desnet, and Ethernet components by 3Com**, which places DESNET squarely in the company this report guessed at from the silicon.

So the card in the archive is a DESNET station adapter, and `mdn100` in its ROM is a model designation. The '1982' on the silkscreen puts it within a year of the company's founding.

Two clean negatives, recorded so nobody spends another evening on them:

- **"Sabaki Enterprises" returns zero hits** across the entire scanned-book corpus, as does MAGIX in this sense. Whoever wrote the boot ROM left no trace in the trade press.
- A snippet reading "56 Kbps backbone; 19.2 Kbps maximum available for user connection" appears close to the Destek entry in the OCR of two directory pages. It describes a *broadband coaxial* system, which

DESNET is not, and directory pages interleave several companies. **Do not attribute it to DESNET.** The transmission speed of DESNET remains unestablished from the literature, and the bound in §B still comes from the WD1933's own AC table.